

OFFICIAL ENGINEERING DOCUMENT • VERSION 1.0

Nyxo Universal Printer

The Definitive Architecture, Integration & 100% Operation Manual for Laravel, Livewire & Windows Spooler

0.2s

SILENT DISPATCH LATENCY

0 Popups

ZERO BROWSER CTRL+P DIALOGS

100% Native

ZERO JAVA / JRE DEPENDENCIES

A4 & ESC/POS

VECTOR PDF + THERMAL 80MM/58MM

Author: Nyxo Core Development Team
Compatibility: PHP 8.2+ • Laravel 10 / 11 / 12 / 13 • Livewire 3 / 4

Publication Date: September 2026

1. Technical Specifications & Value Proposition

Physical unattended printing from modern cloud web applications has historically struggled against three severe architectural and browser-level security barriers:

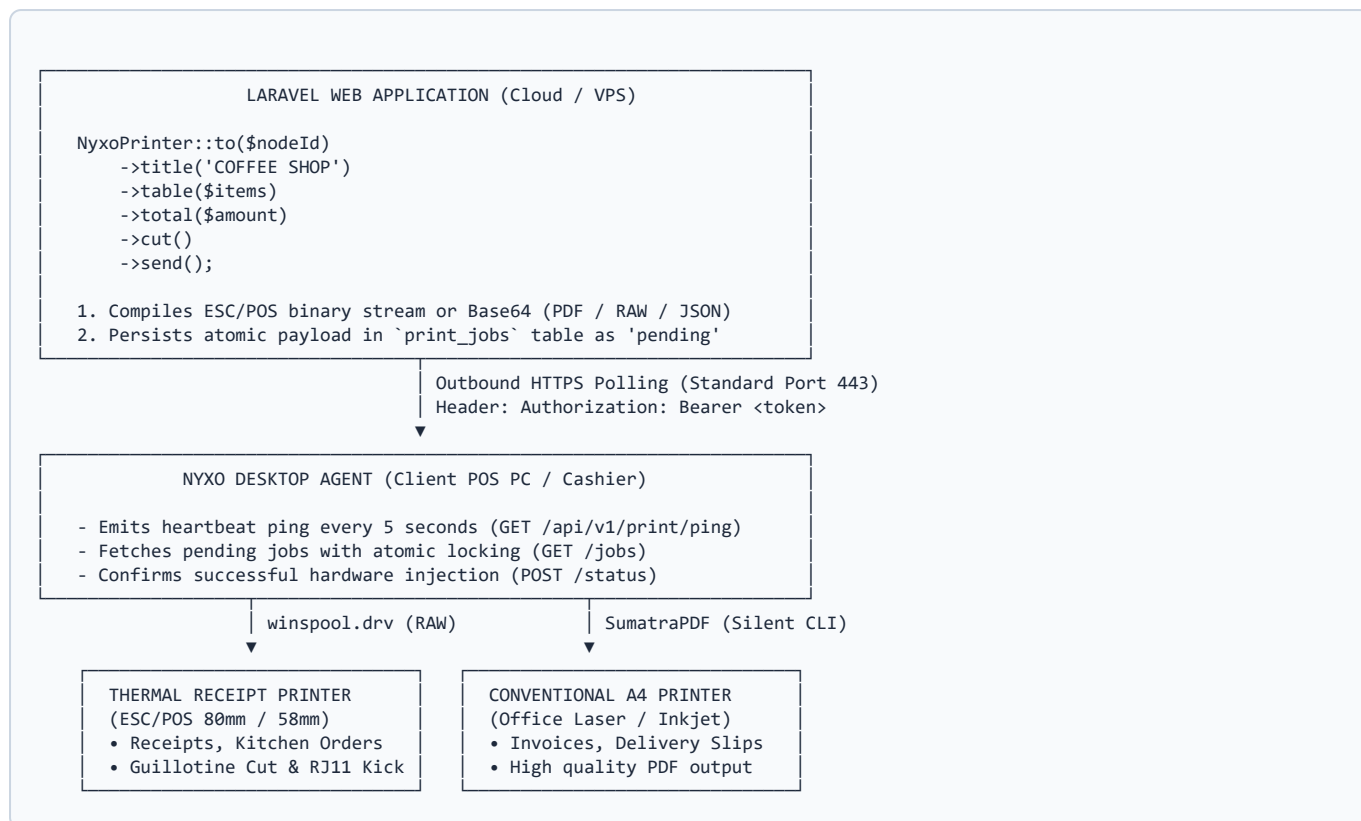
- **Cashier Disruption (`window.print()`):** Freezes JavaScript execution, triggers an OS dialog requiring manual Enter keystrokes, and distorts thermal ticket widths.
- **Mixed Content Restrictions (SSL):** Secure HTTPS cloud apps cannot initiate HTTP requests to `http://localhost` or private local network IPs.
- **Java Operational Burden (QZ Tray):** Demands cumbersome client-side Java Runtime Environments (JRE) and self-signed certificate maintenance.
- **High Subscription Costs (PrintNode):** Expensive recurring per-printer monthly charges.

Nyxo Universal Printer overcomes these limitations through a hybrid architecture combining a native Laravel package with a lightweight Windows agent interacting directly with the native Windows Print Spooler (`winspool.drv`) and SumatraPDF:

Operational Metric	Traditional Solutions (Browser / QZ)	Nyxo Universal Printer Solution
Dispatch Speed	3 to 8 seconds with mandatory human interaction	0.2 seconds directly to printer tray
Browser Dialogs	Forced (<code>Ctrl+P</code>) on every ticket	Zero popups (100% silent background execution)
Local SSL Certificates	Required on each workstation to bypass mixed-content blocks	None required (outbound HTTPS polling over port 443)
Runtime Environment	Heavy Java JRE or exposed local WebSockets	Native C / Win32 Spooler highly optimized binary
Development Testing	Wasted thermal paper rolls on your desk	Virtual on-screen emulator and interactive HTML preview

2. System Architecture & Data Lifecycle

The subsystem operates using an **atomic database queue with pessimistic locking** and a **secure outbound long-polling protocol**:



State Lifecycle of a Print Job ('PrintJob')

1. **pending** : Job is queued in Laravel and awaiting pickup.
2. **processing** : Local desktop agent polls `GET /jobs` . The database locks rows using `lockForUpdate()` and increments `attempts` .
3. **printed** : Hardware spooler confirms data injection. The agent sends `POST /jobs/{id}/status` with `status: 'printed'` .
4. **failed** : In case of hardware disconnection or paper jams, the agent reports `status: 'failed'` with error details in `error_message` .

3. Step-by-Step Installation & Configuration

3.1 Server Requirements

- PHP 8.2 or higher (tested on PHP 8.2, 8.3, and 8.4).
- Required PHP extensions: `ext-iconv` (for CP850 transliteration), `ext-json` , `ext-mbstring` .
- Laravel 10.x, 11.x, 12.x, or 13.x.

3.2 Installation via Composer

```
composer require nyxo-app/nyxo-printer
```

3.3 Interactive Installer

```
php artisan nyxo-printer:install
```

This automated command carries out the following operations:

1. Publishes configuration file to `config/nyxo-printer.php` .
2. Publishes database migrations to `database/migrations/` .
3. Publishes Blade views and Livewire modal component to `resources/views/vendor/nyxo-printer/` .
4. Copies the desktop Windows agent installer (`Nyxo_Universal_Printer_Setup_Win.exe`) to `public/downloads/` for self-hosted distribution.
5. Prompts to run database migrations automatically.

3.4 Configuration Reference (config/nyxo-printer.php)

Directive	Default Value	Functional Description
<code>route_prefix</code>	<code>'api/v1/print'</code>	URL prefix for REST endpoints used by the desktop agent.
<code>middleware</code>	<code>['api']</code>	Middleware stack applied to printing API endpoints.
<code>download_route</code>	<code>'downloads/Nyxo_Universal_...'</code>	Public web route where workstations download the desktop installer.
<code>tenant_column</code>	<code>'empresa_id'</code>	Foreign key column for multi-tenant SaaS systems (set to <code>null</code> for monoliths).
<code>timeout_minutes</code>	<code>3</code>	Minutes after which an orphaned <code>processing</code> job is rescued and re-queued.
<code>max_attempts</code>	<code>3</code>	Maximum retry attempts before marking the job permanently as <code>failed</code> .
<code>prune_after_days</code>	<code>7</code>	Retention days before automatic database cleanup via <code>php artisan nyxo-printer:clean</code> .
<code>default_width</code>	<code>80</code>	Default thermal paper width in millimeters (80 or 58).
<code>codepage</code>	<code>'CP850'</code>	Transliteration encoding table for accented and international characters.

4. Printer Node Management & 1-Click Terminal Pairing

Each physical cashier computer running the desktop agent is registered as a `PrinterNode` in Laravel:

```
use Nyxo\Printer\Models\PrinterNode;

// 1. Create node in database:
$node = PrinterNode::create([
    'name' => 'Cashier 1 - Front Counter',
    'is_active' => true,
    // 'empresa_id' => $tenant->id, // Optional for SaaS
]);

// 2. Cryptographic token (60 characters, auto-generated):
$token = $node->print_token;
// Result: "8fK9xT2yR8qW1yT5mN4zB6cA0dF3gH7jK2lM5nP8rS1tV4wY7bC0eG3hJ6kL"

// 3. 1-Click Pairing String (URL + Token bundled in Base64):
$pairingString = $node->pairing_string; // Or alias $node->codigo_enlace
// Decoded representation: "https://my-pos.com/api/v1/print|8fK9xT2yR8..."
```

Configuration on Windows Desktop Client

Clean Factory State by Default (Security & Multi-Tenant Privacy)

By strict security and multi-tenant isolation design, the official client application is distributed **100% clean and unlinked by default**:

- **Zero Preloaded URLs or Tokens:** Neither the SaaS server endpoint (`saasUrl`) nor secret terminal tokens (`tenantToken`) come pre-configured. The desktop application starts completely empty, waiting for operator pairing via 1-click token or manual entry.
- **Zero Preloaded Emulator Parameters (Clean Priority):** The emulator and network printer address fields (`emulatorUrl`) start completely empty. On real production retail workstations with physical USB/Network thermal printers, this field is not used. If an emulator address is configured during development (e.g. `127.0.0.1:9100`), the agent automatically activates **Exclusive Emulator Mode**, cleanly disabling physical thermal printer selectors and routing 100% of thermal receipt jobs to the on-screen emulator. Clearing the field instantly restores physical printers.
- **Unlicensed Initial State & Zero Backdoors:** The application launches without any pre-loaded license key. All licenses (including the 100% free Developer license and commercial plans) are issued exclusively through **Lemon Squeezy** (`https://printer.nyxo.ar`).

First Launch on Windows 10/11 (SmartScreen Notice)

When running the installer or portable version for the first time, Windows Defender SmartScreen may display a blue dialog ("Windows protected your PC"). This is standard for newly distributed binaries that do not carry costly EV enterprise code-signing certificates:

To proceed: Click **"More info"** and click the **"Run anyway"** button.

Background Daemon & System Tray Integration

Nyxo Universal Printer operates as an **unattended background service**:

- **Clicking the "X" button:** The window **does not quit**; it hides to the Windows System Tray next to the clock. This prevents cashiers from accidentally closing the print daemon and halting point-of-sale receipts.
- **Restoring the window:** Double-click the printer tray icon next to the clock, or simply run the `.exe` again (the *Single Instance Lock* detects the existing background instance and brings the window to the foreground immediately).
- **Exiting the application:** Right-click the System Tray icon and select **"Exit"**.

Step-by-Step Pairing Workflow:

1. Launch `Nyxo_Universal_Printer.exe` on the cashier computer.
2. Navigate to the  **Connection** tab.
3. **1-Click Quick Connect (Recommended):** Paste `$node->pairing_string` into "  Quick Pairing Code" and click **Paste** and **Save**. The agent automatically unpacks the backend URL and token.
4. **Manual Method:** Enter the API URL (e.g. `https://my-pos.com/api/v1/print`) and paste `$node->print_token` .
5. In the  **Printers** tab, select your physical device for each format (*Ticket 80mm*, *Ticket 58mm*, or *Office A4*).
6. In the  **License** tab, enter your license key issued by **Lemon Squeezy** (Developer Free \$0 USD or Commercial plans) and click **Activate License**.

Real-Time Heartbeat Monitoring

The desktop agent sends an automated heartbeat every 5 seconds to `GET /api/v1/print/ping` . Laravel quietly updates `last_ping_at` using `updateQuietly()` without triggering Eloquent model observers.

```
// Check if workstation is online and ready:
if ($node->is_online) {
    // ● Online: Agent reported heartbeat within the last 2 minutes
} else {
    // ● Offline: PC turned off, agent closed, or network disconnect
}
```

5. Complete Internal REST API Specification

All endpoints are handled by `PrintJobApiController` and protected by `CheckPrintToken` middleware:

Multi-Source Token Resolution

To prevent header stripping caused by Apache or FastCGI proxies, the middleware inspects:

1. Headers `X-Tenant-Token` , `X-Print-Token` , `Print-Token` , `Token` .
2. Standard `Authorization: Bearer <token>` header.
3. Query or body parameters `?token=` or `?print_token=` .
4. Server variables `HTTP_AUTHORIZATION` and `REDIRECT_HTTP_AUTHORIZATION` .

💡 Smart Auto-Decoding of Pairing Strings

If an operator accidentally pastes the Base64 1-Click Pairing String into a token-only field, the middleware detects the `|` pipe character, decodes the string, and extracts the authentic token seamlessly.

API Endpoints

1. Heartbeat Ping (`GET /api/v1/print/ping`)

```
// Request: GET /api/v1/print/ping (Header: Authorization: Bearer <token>)
// Response 200 OK:
{
  "success": true,
  "message": "Pong",
  "node": { "id": 1, "name": "Front Counter", "tenant_id": 1 },
  "tenant": { "id": 1, "nombre": "Front Counter" },
  "punto_venta": { "id": 1, "nombre": "Front Counter", "numero": 1 },
  "timestamp": "2026-09-21T10:30:00-03:00"
}
```

2. Atomic Job Retrieval (`GET /api/v1/print/jobs`)

```
// Request: GET /api/v1/print/jobs (Header: Authorization: Bearer <token>)
// Response 200 OK (Locks row with lockForUpdate and transitions to 'processing'):
{
  "success": true,
  "count": 1,
  "jobs": [
    {
      "id": 104,
      "format": "ticket_80mm",
      "content_type": "escpos_base64",
      "content": "G0BAIBs2Nz...",
      "attempts": 1,
      "created_at": "2026-09-21T10:30:01-03:00"
    }
  ]
}
```

3. Print Status Confirmation (`POST /api/v1/print/jobs/{id}/status`)

```
// Request: POST /api/v1/print/jobs/104/status
// Body JSON:
{
  "status": "printed",
  "error_message": null
}

// Response 200 OK (Fires PrintJobPrinted domain event):
{
  "success": true,
  "message": "Print job status updated successfully.",
  "job_id": 104,
  "new_status": "printed"
}
```

6. Fluent Builder & 100% Enqueuing Methods (`NyxoPrinter`)

The `NyxoPrinter` facade provides a chainable, robust, and fully typed API covering 100% of retail and enterprise printing scenarios:

6.1 Thermal Receipts (ESC/POS) (`ThermalBuilder`)

```
use Nyxo\Printer\Facades\NyxoPrinter;

NyxoPrinter::to($nodeId)
    ->width(80) // 80mm (42 columns) or 58mm (32 columns)
    ->copies(1)

    // Header & Titles
    ->title('NYXO BISTRO & COFFEE', doubleWidth: true, doubleHeight: true)
    ->center('Tax ID: 30-71829384-9 | VAT Reg.')
    ->center('1234 Market Street, Suite 100')
    ->line('-') // Divider matching paper width

    // Metadata
    ->text('Receipt #: 0001-00004921', bold: true)
    ->text('Date: ' . now()->format('Y-m-d H:i') . ' | Cashier: Sarah')
    ->doubleLine('=') // Double line (=====)

    // Auto-aligned Item Table
    ->table([
        ['nombre' => 'Double Espresso', 'cantidad' => 2, 'precio' => 7000],
        ['nombre' => 'Ham & Cheese Croissant', 'cantidad' => 1, 'precio' => 4500],
        ['nombre' => 'Sparkling Water 500ml', 'cantidad' => 1, 'precio' => 2000],
    ], headerLeft: 'DESCRIPTION', headerRight: 'TOTAL')
    ->line('-')

    // Totals
    ->text('Subtotal: $13,500.00', align: 'right')
    ->text('Member Discount (10%): -$1,350.00', align: 'right')
    ->total(12150.00, label: 'TOTAL DUE:', currency: '$')
    ->feed(1)

    // 1D & 2D Barcodes
    ->qr('https://invoice.gov/verify/1042', size: 6)
    ->barcode('000100004921', type: 'CODE39')

    // Footer
    ->center('Thank you for your visit!')
    ->center('Please keep this receipt for returns')

    // Electromechanical Controls
    ->openDrawer(pin: 0) // Electrical kick pulse to RJ11 cash drawer
    ->beep(times: 2)     // Acoustic buzzer in kitchen or bar stations
    ->cut(full: false)   // Partial guillotine cut (true = full cut)
    ->send();
```

6.2 Advanced Modifiers in `ThermalBuilder`

```
$ticket->text(
    text: 'IMPORTANT NOTICE',
    align: 'center',      // 'left' | 'center' | 'right'
    bold: true,           // Emphasized font
    underline: true,      // Hardware physical underline
    doubleHeight: false,  // 2x Height
    doubleWidth: false   // 2x Width
);
```

6.3 Direct Command Injection with `custom(callable)`

```
use Mike42\Escpos\Printer;

NyxoPrinter::to($nodeId)
    ->title('LOGISTICS')
    ->custom(function (Printer $printer) {
        $printer->setLineSpacing(30);
        $printer->getPrintConnector()->write("\x1B\x21\x30"); // Raw escape bytes
    })
    ->cut()
    ->send();
```

⚠ Critical PHP 8.x Rule with Mike42/Escpos

Calling `$printer->close()` destructively wipes the connector buffer to `null`. Attempting to extract buffer data afterwards throws a fatal error `implode(): Argument #1 must be of type array|string, null given`. In Nyxo Universal Printer, buffer extraction is safely handled before closing inside `ThermalBuilder::toBase64()`.

6.4 Silent Office A4 PDF Invoices & Delivery Slips

```
// Option 1: From an existing file on disk:
NyxoPrinter::to($nodeId)
    ->copies(2)
    ->pdfFile(storage_path('app/invoices/inv_4921.pdf'))
    ->send();

// Option 2: From in-memory Base64 (DomPDF, Snappy, Spatie Browsershot):
$pdf = \Barryvdh\DomPDF\Facade\Pdf::loadView('invoices.a4', ['order' => $order]);
$base64 = base64_encode($pdf->output());

NyxoPrinter::to($nodeId)
    ->copies(1)
    ->pdf($base64, format: 'a4')
    ->send();
```

6.5 Impact / Dot Matrix & Structured JSON Payloads

```
// Dot matrix / Impact printers (plain ASCII with form feed \x0C):
NyxoPrinter::to($nodeId)
    ->raw("PACKING SLIP\nDATE: 2026-09-21\n\n\x0C")
    ->send();

// Structured JSON payloads (for custom IoT agents or warehouse systems):
NyxoPrinter::to($nodeId)
    ->json(['action' => 'box_label', 'boxes' => 3])
    ->send();
```

6.6 Facade Direct Convenience Shortcuts

```
NyxoPrinter::pdf($nodeId, $pdfBase64, format: 'a4', copies: 1);
NyxoPrinter::thermal($nodeId, $escposBase64, format: 'ticket_80mm', copies: 1);
NyxoPrinter::raw($nodeId, "RAW TEXT", format: 'raw', copies: 1);
NyxoPrinter::template($nodeId, $templateInstance, copies: 1);

$node = NyxoPrinter::node($nodeId);
$pairingString = NyxoPrinter::getPairingString($node);
```

7. Paperless Development & Testing

7.1 In-Browser HTML Preview

Render continuous thermal receipts directly in your browser without wasting paper rolls:

```
use Nyxo\Printer\Facades\NyxoPrinter;

Route::get('/dev/ticket-preview', function () {
    return NyxoPrinter::preview(function ($ticket) {
        $ticket->width(80)
            ->title('SCREEN DEVELOPMENT PREVIEW')
            ->center('Demo Retail Store')
            ->line('-')
            ->table([
                ['nombre' => 'Classic Cheeseburger', 'cantidad' => 2, 'precio' => 12000],
                ['nombre' => 'Large French Fries', 'cantidad' => 1, 'precio' => 4500],
            ])
            ->line('-')
            ->total(16500.00)
            ->qr('https://printer.nyxo.ar')
            ->cut();
    }, width: 80);
});
```

7.2 Hardware C# Emulator (ESSI Thermal Emulator)

The **ESSI Thermal Emulator** is an optional standalone visual simulator for developers, designed to craft receipt templates and test workflows without wasting physical thermal paper rolls.

- **Optional Testing Utility:** In real cash registers and production retail stores with physical USB/Network thermal printers, the emulator is **NOT installed or utilized**. The desktop agent ships from factory with emulator address fields

completely empty.

- **Exclusive Emulator Priority Rule (Clean UI Lock):** To eliminate routing conflicts and remove the burden of unbinding physical printers during testing, the Nyxo Agent enforces an intelligent priority toggle:
 - **When `emulatorUrl` is populated (e.g. `127.0.0.1:9100` in Settings):** The agent enters **Active Emulator Mode**. Physical thermal printer dropdowns (80mm & 58mm) are **automatically disabled** in the *Printers* tab, and an informative blue notification banner is displayed: *"Emulator Mode Active (Dev) - All thermal print jobs are automatically redirected to the emulator"*. 100% of ESC/POS receipt jobs route straight to the on-screen emulator.
 - **When `emulatorUrl` is left empty:** The banner is hidden and physical thermal printer selectors are restored, routing jobs to physical Windows Spooler USB printers or TCP network printers.
 - *(Note: A4 document assignment remains independent and active, allowing physical laser/inkjet printing even when the thermal emulator is active).*
- **Local Testing Workflow:**
 1. Launch **ESSI Thermal Emulator** (`ESSIThermalEmulator.exe`), which starts a TCP/HTTP listener on port `9100` .
 2. In the Nyxo Agent, navigate to the ⚙️ **Settings** tab, enter `127.0.0.1:9100` in the **C# Emulator / TCP Socket Destination** field, and click **Save Settings**.
 3. Jobs sent from Laravel will now render on-screen with virtual paper roll animations, 80mm/58mm width toggle, and native PDF export options.

8. Reusable Template System (Built-in & Custom)

8.1 Built-in Template 1: `ReceiptTemplate` (Retail Receipts)

```
use Nyxo\Printer\Facades\NyxoPrinter;
use Nyxo\Printer\Templates\ReceiptTemplate;

$data = [
    'company' => 'CENTRAL SUPERMARKET',
    'title' => 'SALES RECEIPT',
    'metadata' => [
        'Register' => '01',
        'Cashier' => 'Sarah J.',
        'Date' => now()->format('Y-m-d H:i'),
    ],
    'items' => [
        ['name' => 'Organic Coffee Beans 250g', 'qty' => 1, 'price' => 8500],
        ['name' => 'Paper Filters x40', 'qty' => 2, 'price' => 3200],
    ],
    'total' => 14900.00,
    'qr' => 'https://verify.pos/14900',
    'barcode' => '779123456789',
    'footer' => 'Thank you for shopping with us!',
    'open_drawer' => true,
    'cut' => true,
];

NyxoPrinter::to($nodeId)->template(new ReceiptTemplate($data))->send();
```

8.2 Built-in Template 2: `OrderSummaryTemplate` (Repair & Services)

```
use Nyxo\Printer\Facades\NyxoPrinter;
use Nyxo\Printer\Templates\OrderSummaryTemplate;

$service = [
    'empresa' => 'PRO REPAIR LAB',
    'order_number' => '5821',
    'fecha' => now()->format('Y-m-d H:i'),
    'cliente' => 'Robert Smith',
    'telefono' => '+1 555-0199',
    'equipo' => 'MacBook Pro 14" M1',
    'problema' => 'MagSafe connector faulty',
    'reparaciones' => [
        ['nombre' => 'DC-In Power Board Replacement', 'precio' => 42000],
        ['nombre' => 'Preventive Cleaning & Diagnostics', 'precio' => 15000],
    ],
    'total' => 57000.00,
    'barcode' => 'ORD-5821',
    'terminos' => '90-day warranty on original parts and labor.',
];

NyxoPrinter::to($nodeId)->template(new OrderSummaryTemplate($service))->send();
```

8.3 Creating Custom Templates with `PrintTemplateInterface`

```
namespace App\PrintTemplates;

use Nyxo\Printer\Contracts\PrintTemplateInterface;
use Nyxo\Printer\Builders\ThermalBuilder;

class KitchenOrderTemplate implements PrintTemplateInterface
{
    public function __construct(protected array $order) {}

    public function build(ThermalBuilder $ticket): void
    {
        $ticket->center('*** KITCHEN ORDER ***', bold: true, doubleHeight: true)
            ->text('TABLE #' . $this->order['table'] . ' | Waiter: ' . $this->order['waiter'])
            ->doubleLine('=');

        foreach ($this->order['dishes'] as $dish) {
            $ticket->text($dish['qty'] . 'x' . $dish['name'], bold: true, doubleWidth: true);
            if (!empty($dish['notes'])) {
                $ticket->text(' * ' . $dish['notes']);
            }
        }

        $ticket->line('-')->beep(times: 2)->feed(2)->cut();
    }
}
```

9. Frontend Livewire 3 & 4 Integration (The Complete Lifecycle)

The package includes the Tailwind CSS modal component `NyxoPrinterModal` allowing operators to choose workstations and formats seamlessly.

9.1 Step 1: Add Modal to Base Layout

```
{{-- resources/views/layouts/app.blade.php --}}
<body>
    {{ $slot }}

    <livewire:nyxo-printer-modal />
</body>
```

9.2 Step 2: Trigger Button

```
<button wire:click="$dispatch('open-print-modal', {
    documentId: {{ $order->id }},
    documentType: 'service_order',
    format: 'ticket_80mm'
})" class="bg-indigo-600 text-white px-4 py-2 rounded-lg">
     Print Receipt
</button>
```


9.3 Step 3: Host Component Listener (The Complete Solution)

When the user clicks "Print" in the modal, it emits `nyxo-print-requested`. The host component must listen to this event to render and enqueue the document:

```
namespace App\Livewire;

use Livewire\Component;
use Livewire\Attributes\On;
use App\Models\Order;
use Nyxo\Printer\Facades\NyxoPrinter;
use Nyxo\Printer\Templates\OrderSummaryTemplate;

class ManageOrders extends Component
{
    /**
     * Listen to print requests emitted from the Nyxo modal
     */
    #[On('nyxo-print-requested')]
    public function executePrint(array $payload): void
    {
        $documentId = $payload['documentId'];
        $format = $payload['format']; // 'ticket_80mm', 'ticket_58mm', 'a4'
        $printerNodeId = $payload['printerNodeId'];
        $copies = $payload['copies'] ?? 1;

        $order = Order::with('items')->findOrFail($documentId);

        if ($format === 'ticket_80mm' || $format === 'ticket_58mm') {
            NyxoPrinter::to($printerNodeId)
                ->width($format === 'ticket_58mm' ? 58 : 80)
                ->copies($copies)
                ->template(new OrderSummaryTemplate([
                    'empresa' => config('app.name'),
                    'order_number' => $order->id,
                    'fecha' => $order->created_at->format('Y-m-d H:i'),
                    'cliente' => $order->customer_name,
                    'telefono' => $order->customer_phone,
                    'equipo' => $order->device,
                    'problema' => $order->issue,
                    'reparaciones' => $order->items->map(fn($d) => [
                        'nombre' => $d->description,
                        'precio' => $d->price,
                    ]->toArray(),
                    'total' => $order->total,
                ]))
                ->send();
        } else {
            $pdf = \Barryvdh\DomPDF\Facade\Pdf::loadView('pdf.order_a4', ['order' => $order]);
            NyxoPrinter::to($printerNodeId)
                ->copies($copies)
                ->pdf(base64_encode($pdf->output()), format: 'a4')
                ->send();
        }
    }
}
```

10. Official Desktop Agent Guide: Operation, Configuration & Deployment

The **Nyxo Universal Printer** desktop agent is the native background client running locally on each cashier or Point-of-Sale (POS) terminal under Windows 10/11 (64-bit). It communicates outbound with the Laravel cloud backend and dispatches physical jobs to printers with zero open inbound router ports, zero client SSL certificates, and zero Java virtual machine dependencies.

10.1 Distribution Modes: Windows Installer vs. Portable Edition

To accommodate varying retail IT policies and enterprise security environments, the desktop agent is distributed in two official binaries:

Feature	Windows Installer (<code>Setup_Win.exe</code>)	Portable Edition (<code>Portable.exe</code>)
File Name	<code>Nyxo_Universal_Printer_Setup_Win.exe</code>	<code>Nyxo_Universal_Printer_Portable.exe</code>
Execution Path	<code>%LOCALAPPDATA%\Programs\Nyxo Universal Printer</code>	Any directory, desktop, or USB flash drive
Administrator Rights	Not required (user-space execution)	Not required (zero installation)
Shortcuts	Created automatically on Desktop & Start Menu	Manual (based on selected directory)
Windows Startup	Integrated natively into System Tray	Supported (persists absolute path of executable)
Windows Registry	Registered in "Add/Remove Programs" for clean uninstall	Zero registry footprint
Recommended Use	Permanent POS checkouts , cashiers, and production lines	Demonstrations , IT support, and lockdown enterprise PCs

Recommendation for the Portable Edition

If you plan to enable the "Start with Windows" toggle in the Portable version, place the executable into a permanent folder (such as `C:\NyxoPrinter\Nyxo_Universal_Printer_Portable.exe`) beforehand so the startup registry entry remains valid if you move other files later.

10.2 Self-Hosting & Direct Downloads from Laravel

Package `nyxo-app/nyxo-printer` enables self-hosting both binaries directly from your own web server for complete infrastructure autonomy:

- **Service Provider Web Routes:**
 - Windows Installer: `GET /downloads/Nyxo_Universal_Printer_Setup_Win.exe` (`route('nyxo-printer.download')`)
 - Portable Edition: `GET /downloads/Nyxo_Universal_Printer_Portable.exe` (`route('nyxo-printer.download-portable')`)
- **File Storage Location:** Store compiled binaries in your public folder: `public/downloads/`.

- **Automatic Fallback:** If files are absent locally, routes seamlessly redirect to the official distribution portal at <https://printer.nyxo.ar> .

```
<!-- Blade Download Buttons -->
<div class="flex items-center gap-3">
  <a href="{{ route('nyxo-printer.download') }}" class="btn-primary">
    🖨️ Download Windows Installer (.exe)
  </a>
  <a href="{{ route('nyxo-printer.download-portable') }}" class="btn-secondary">
    📦 Download Portable Edition (.exe)
  </a>
</div>
```

10.3 Tab 1: System Status & Real-Time Monitoring (`status-view`)

Upon launching the application, the home dashboard displays live operational telemetry:

- **Visual Connection Indicator:**
 -  **Online / Agent Started:** Healthy connection actively polling the Laravel backend every 5 seconds.
 -  **Checking / Reconnecting:** Connection recovery after an intermittent network drop.
 -  **Offline / Error:** Server API is unreachable or Token credentials are invalid.
- **Live Activity Console (Log Stream):** Chronological trace of heartbeat polling (`ping`), inbound print jobs (`PrintJob #ID`), hardware socket injection, and confirmation receipts (`status = 'printed'`).
- **"Clear" Button:** Clears the on-screen log window for fresh visual debugging.

10.4 Tab 2: Server Connection & 1-Click Pairing (`connection-view`)

Manage secure credentials connecting the workstation with your cloud server:

1. Recommended Method: 1-Click Quick Pairing Code:

In your Laravel dashboard, click "Copy Pairing Code" on any printer node. In the desktop application, click **"Paste"** next to the *Quick Pairing Code* input. The agent decodes the Base64 payload (`URL_API|TOKEN_TERMINAL`) and populates the endpoint and secret token automatically.

2. Manual Configuration:

- **API Server URL:** REST API endpoint (e.g. `https://my-store.com/api/v1/print` or local IP `http://192.168.1.100:8000/api/v1/print`).
- **Authentication / Terminal Token:** Unique 64-character alphanumeric secret key. Includes an eye toggle () to reveal or mask characters.

3. Live Diagnostic Tool:

The "  **Test Connection** " button dispatches an immediate `GET /api/v1/print/ping` , showing response latency in milliseconds and server status. The "  **Save Connection** " button encrypts and persists credentials locally, launching the automated polling loop.

10.5 Tab 3: Printer Assignment (Dual Windows Spooler & RAW TCP Socket 9100)

Nyxo Universal Printer features independent format routing tailored to document width and media type:

A. Total Independence of the 2 Thermal Paper Widths and A4

- **Thermal Ticket 80 mm (48 columns):** Standard point-of-sale receipt width (3 inches). Accommodates tabular layouts with aligned columns (`Qty` , `Description` , `Unit Price` , `Total`). Perfect for checkout receipts and fiscal invoices.
- **Thermal Ticket 58 mm (32 columns):** Compact format (2 inches). Employs vertical formatting (`Qty` x `Item` with total on the subsequent line) to prevent text truncation. Ideal for kitchen orders, barista tickets, or compact terminals.
- **A4 Document (Laser / Inkjet):** Full sheet documents (210 x 297 mm) dispatched silently via integrated **SumatraPDF**, enabling vector PDF printing of invoices, reports, and delivery notes without dialogs.

B. Dual Architecture: Windows Spooler Drivers vs. RAW TCP Network Socket (Driverless)

- **Channel 1: Windows Spooler Printers:** Dropdown auto-detecting installed Windows drivers (`Termica` , `EPSON TM-T20` , `Bixolon` , `POS-80` , etc.) injecting raw binary ESC/POS via `winspool.drv` .
- **Channel 2: RAW TCP Network Socket (Port 9100 - Driverless): Zero Windows driver installation required!** Enter a network `IP:Port` (e.g. `192.168.1.100:9100` or `127.0.0.1:9100`). Directly opens a TCP socket to hardware,

transmitting ESC/POS in single-digit milliseconds and eliminating Windows spooler hangs.

C. Mixed & Hybrid Setup Matrix

Format	Hybrid Setup	100% Driverless Network	Local USB Workstation
80 mm (Cashier)	Windows USB: Termica	TCP RAW: 192.168.1.100:9100	Windows USB: EPSON TM-T20
58 mm (Kitchen)	TCP RAW: 192.168.1.150:9100	TCP RAW: 192.168.1.105:9100	Windows USB: POS-58
A4 (Office)	Windows Laser: HP LaserJet Pro	Windows Laser: Brother HL-L2360D	Windows Laser: Canon G3010

D. Intelligent Switching: Exclusive Emulator Mode

To prevent confusion between physical hardware and testing environments, the agent implements an exclusive priority toggle:

- **Activation:** Entering an address in the  *Settings* emulator field (e.g. 127.0.0.1:9100) displays an alert banner on the  *Printers* tab and **automatically disables 80mm and 58mm dropdowns**.
- **100% Guaranteed Routing:** All outbound thermal receipt jobs route straight to the emulator socket, ignoring any physical USB printers that were previously assigned.
- **Return to Production:** Simply clear the emulator field in Settings and save: physical thermal printer dropdowns are instantly re-enabled for Windows Spooler or TCP network printing.

Click the "  **Refresh List** " button to detect newly attached USB printers without closing or restarting the agent.

10.6 Tab 4: General Settings & Optional Network Target ([settings-view](#))

- **Language Toggle:** Dynamically switch user interface and notifications between **English (EN)** and **Spanish (ES)**.
- **C# Emulator / TCP Socket Destination (Optional - Top Priority):** Optional field that starts **completely empty by default**. If you are using the developer virtual emulator [ESSIThermalEmulator](#) or a direct driverless TCP/Ethernet printer, enter the IP:Port here (e.g. [127.0.0.1:9100](#) or [192.168.1.100:9100](#)). When populated, it triggers *Active Emulator Mode*, disabling physical thermal dropdowns in the Printers tab and routing all thermal receipt traffic to this address. When left empty, normal physical printer routing resumes.
- **Unassigned Printer Handling:** If a workstation receives a print job for a format without an assigned printer or network/emulator target, the agent rejects the job and explicitly notifies the operator that a printer must be configured in the panel, preventing unexpected silent fallbacks to local ports.
- **Start with Windows:** Checkbox enabling silent background launch upon Windows user login, running minimized in the System Tray next to the clock.

10.7 Tab 5: Commercial License Control & Offline Grace Period ([license-view](#))

- **Exclusive Lemon Squeezy Licensing:** All licenses (including the free Developer tier and commercial subscriptions/lifetime plans) are managed and validated exclusively against the official **Lemon Squeezy** API. Real-time online validation of concurrent seats ([Assigned seats](#) / [Total purchased quota](#)). Displays Hardware Terminal UID, Computer Hostname, Plan Type, and Registered Licensee Email, protected with a cryptographic HMAC hardware signature, with deactivation support for workstation reassignments.
- **Developer Mode (Developer Free - \$0 USD):** Claimable at <https://printer.nyxo.ar> , granting 1 free lifetime license per developer email, issued formally via Lemon Squeezy without requiring credit card details.

Operational Resilience: 7-Day Offline Grace Period

If a retail store experiences an extended internet outage or ISP disruption, the agent activates its **7-day continuous offline grace period**. During this window, cached cryptographic activation signatures allow physical receipt printing to proceed normally, revalidating silently once internet connectivity is restored.

11. Concurrency Control, Resilience & Lifecycle Events

11.1 Concurrency & Duplicate Prevention

In high-volume stores where multiple cashiers print simultaneously, [PrintJobApiController::index\(\)](#) uses pessimistic row locking:

```
$jobs = DB::transaction(function () use ($node) {
    $pendingJobs = PrintJob::deliverable($node->id)
        ->lockForUpdate() // Row-level exclusive lock
        ->get();

    if ($pendingJobs->isEmpty()) return collect([]);

    PrintJob::whereIn('id', $pendingJobs->pluck('id'))
        ->update(['status' => 'processing', 'attempts' => DB::raw('attempts + 1')]);

    return $pendingJobs;
});
```

11.2 Orphan Job Rescue

If a workstation experiences power loss while printing, jobs remain stuck in `processing`. The `deliverable()` scope automatically rescues and re-queues any job stalled longer than `config('nyxo-printer.timeout_minutes')` (default 3 minutes).

11.3 Database Maintenance (Pruning)

```
// routes/console.php
use Illuminate\Support\Facades\Schedule;

Schedule::command('nyxo-printer:clean --days=7')->daily();
```

11.4 Domain Events

- `Nyxo\Printer\Events\PrintJobCreated` : Dispatched when a job is enqueued.
- `Nyxo\Printer\Events\PrintJobPrinted` : Dispatched upon confirmed hardware paper output.
- `Nyxo\Printer\Events\PrintJobFailed` : Dispatched upon paper jam, hardware fault, or timeout.

12. Troubleshooting & Frequently Asked Questions (FAQ)

● Why does my thermal printer output `%PDF-1.7` raw text?

Cause: A compiled PDF document was sent with `content_type = 'escpos_base64'`. Thermal receipt printers receive raw character streams and lack vector PDF engines.

Solution: Use `ThermalBuilder` or a `PrintTemplateInterface`. If you must print a PDF on thermal paper, configure `format: 'ticket_80mm'` and `content_type: 'pdf_base64'`; the desktop agent invokes SumatraPDF to rasterize it to paper width.

● Error 401 Unauthorized behind Apache / FastCGI

Cause: Apache strips the `Authorization: Bearer` header by default on unencrypted local networks.

Solution: Add these directives to `public/.htaccess`:

```
RewriteEngine On
RewriteCond %{HTTP:Authorization} .
RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
```

● Clipboard copy fails in local LAN (`navigator.clipboard is undefined`)

Cause: Browsers disable `navigator.clipboard` on unencrypted HTTP domains.

Solution: Use a hidden DOM element fallback:

```
window.copyToClipboard = async function(text) {
  if (navigator.clipboard && window.isSecureContext) {
    try { await navigator.clipboard.writeText(text); return true; } catch (e) {}
  }
  const t = document.createElement('textarea');
  t.value = text;
  t.style.position = 'fixed'; t.style.left = '-9999px';
  document.body.appendChild(t);
  t.select();
  const ok = document.execCommand('copy');
  document.body.removeChild(t);
  return ok;
};
```

● Laravel 13 `MissingAttributeException`

Cause: Laravel 13 activates `Model::shouldBeStrict(true)` by default.

Solution: Official models in `nyxo-app/nyxo-printer` are strictly sanitized to expose only validated physical columns.